Fake News Detection System: Noisy Data Technical Report

1. Project Architecture & Setup

The project is built as an end-to-end Machine Learning pipeline entirely in Python, utilizing standard data science and ML libraries:
- Data Manipulation: pandas, numpy
- Natural Language Processing (NLP): nltk, re (Regex)
- Machine Learning Engine: scikit-learn
- Visualization: matplotlib, seaborn, wordcloud

The workflow follows a sequential architecture:
Data Ingestion -> Feature Engineering -> Preprocessing -> TF-IDF Vectorization -> Model Training & Tuning -> Evaluation.

2. Data Preprocessing Techniques

Since raw news articles contain noise (like URLs, punctuation, and varying casing), the text goes through a rigorous NLP cleaning process before feeding it to the model.

1. Feature Merging: The article 'title' and 'text' (body) are concatenated into a single string. This is crucial because fake news often relies on sensational clickbait titles.
2. URL & HTML Stripping: Regular expressions (Regex) are used to remove web links and residual HTML tags.
3. Lowercasing & Punctuation Removal: All text is converted to lowercase, and non-alphanumeric characters are stripped out.
4. Stopword Removal: Common English words that do not carry significant meaning (e.g., 'the', 'is', 'in') are removed using the NLTK library.
5. Lemmatization: Words are reduced to their dictionary root form (e.g., 'running' becomes 'run'). This normalizes the vocabulary.

3. Data Splitting & TF-IDF Vectorization

Once the text is cleaned, it must be converted into numbers because ML models cannot read text natively.

- Data Split: The dataset (44,898 articles) is split into a Train Set (80%) and a Test Set (20%). A stratified split is used to ensure the exact same ratio of Real to Fake news exists in both sets.
- TF-IDF Vectorization (Term Frequency-Inverse Document Frequency): The training text is converted into a massive mathematical matrix (35,911 rows x 10,000 columns).
  * TF: Counts how often a word appears in an article.
  * IDF: Penalizes words that appear across almost all articles.
  * N-Grams: The vectorizer looks at single words (Unigrams) and pairs of words (Bigrams).

4. Model Training, Epochs, & Time

How many Epochs?
The concept of 'epochs' applies to Neural Networks (like Deep Learning models), which iteratively loop over the dataset to update weights. Naive Bayes models do not use epochs. Instead, they calculate probabilities algebraically in a single pass over the dataset. This makes them significantly faster and highly efficient.

Training Process:
1. MultinomialNB (Baseline): Trained in a single pass.
2. ComplementNB: Trained as our primary model. It handles class imbalances better.
3. Hyperparameter Tuning: We used GridSearchCV combined with 5-Fold Cross-Validation. The model trained itself 5 different times on different subsets across various smoothing parameters (alpha). It automatically discovered that alpha=0.01 yielded the best results.

Training Time:
Because Naive Bayes relies on simple probability counting rather than complex matrix backpropagation, it is blazingly fast. The entire pipeline took approximately 3 minutes and 50 seconds running entirely on a standard CPU. The actual model training portion of that time was completed in just a few seconds.

5. How the ML Models Work Behind the Scenes

We used Naive Bayes, which is rooted in Bayes' Theorem of probability.

1. The 'Naive' Assumption
It assumes that every word in an article is completely independent of the others. Even though this is linguistically false, it mathematically works incredibly well for text classification.

2. Calculating Probabilities (Learning)
During training, the model simply counts word frequencies and calculates probabilities (e.g., probability of 'Shocking' given it is Fake news).

3. Making Predictions (Inference)
When a brand new article is passed into the model, it breaks the article down into its individual words. It then calculates two final scores by multiplying known probabilities together. Whichever score is mathematically higher wins, and the model confidently labels the article as Fake or Real!

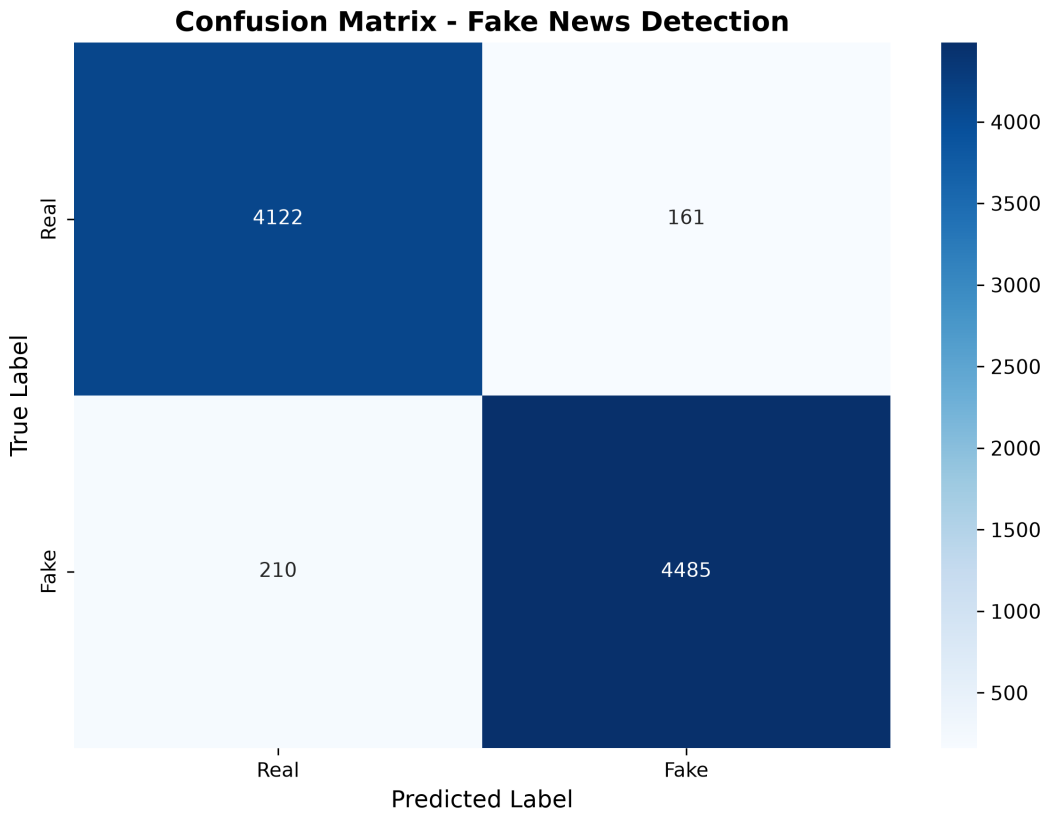
6. Results & Visualizations

Final Model Comparison (Terminal Output)

```
============================================================
           MultinomialNB  ComplementNB  Best ComplementNB
accuracy          0.9529        0.9523             0.9587
precision         0.9527        0.9521             0.9584
recall            0.9529        0.9525             0.9588
f1                0.9528        0.9523             0.9586
roc_auc           0.9889        0.9889             0.9911
============================================================
```

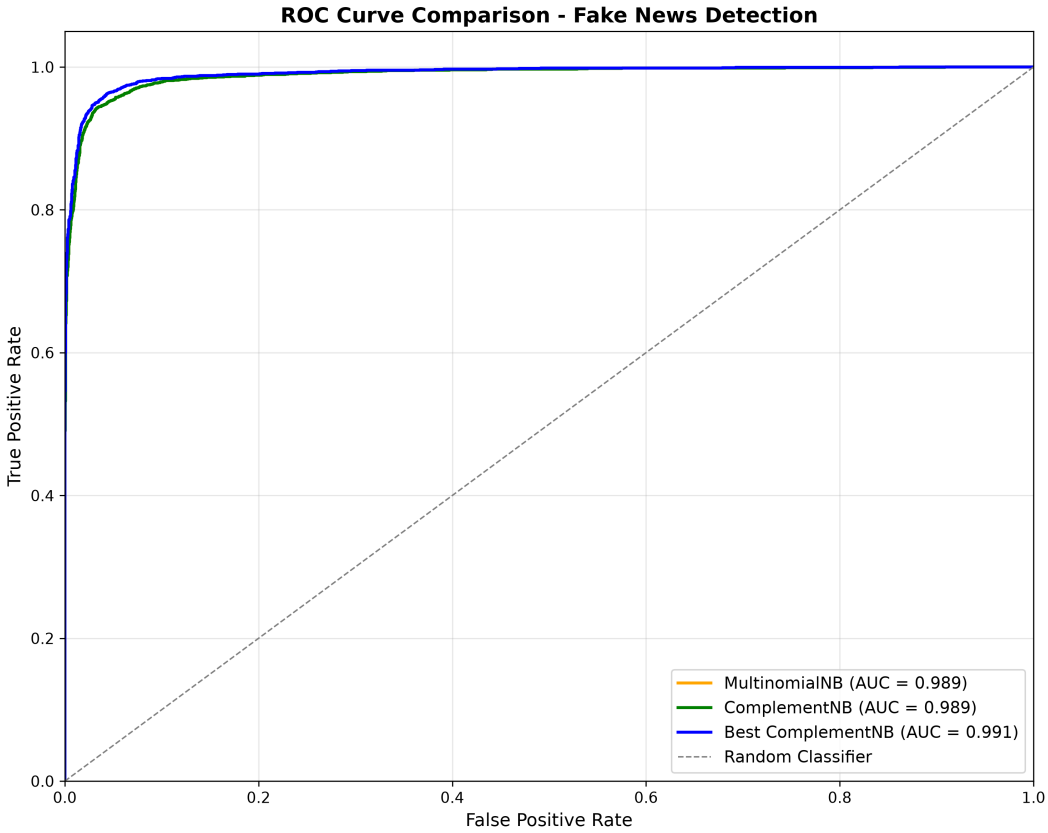
The Best ComplementNB model achieved an Accuracy of 95.87%, Precision of 95.84%, Recall of 95.88%, F1-Score of 0.9586, and ROC-AUC of 0.9911. Below are the visual evaluation matrices and graphs.

Confusion Matrix (Noisy Model)



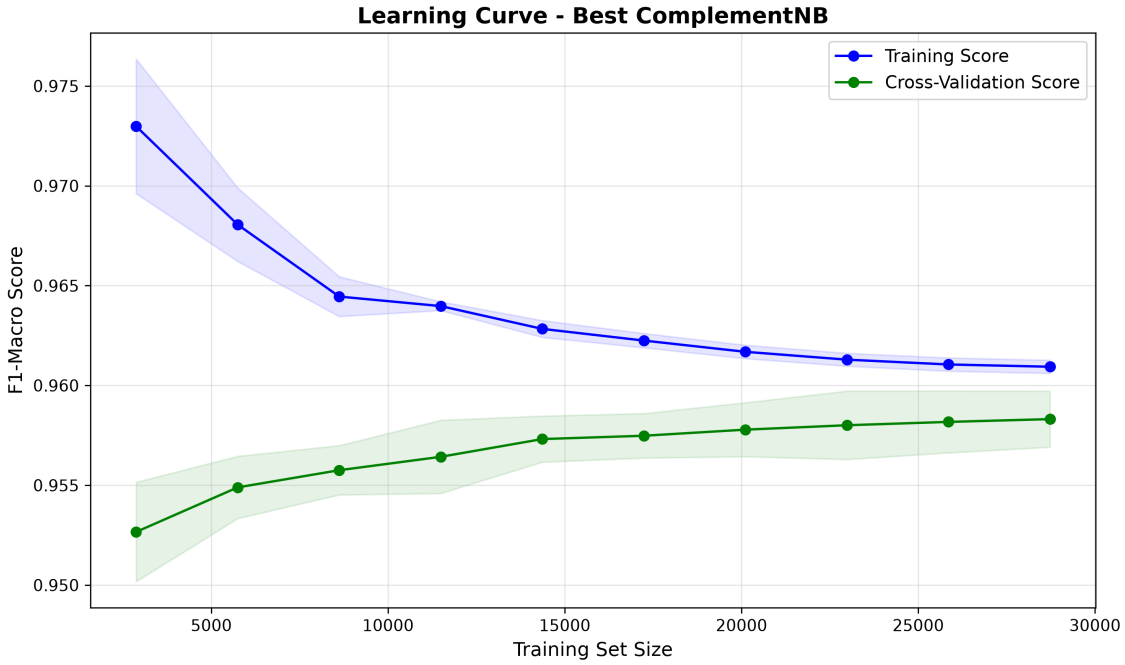
ROC Curve & AUC Scores (Noisy Model)

Fake News Detection System: Noisy Data Technical Report

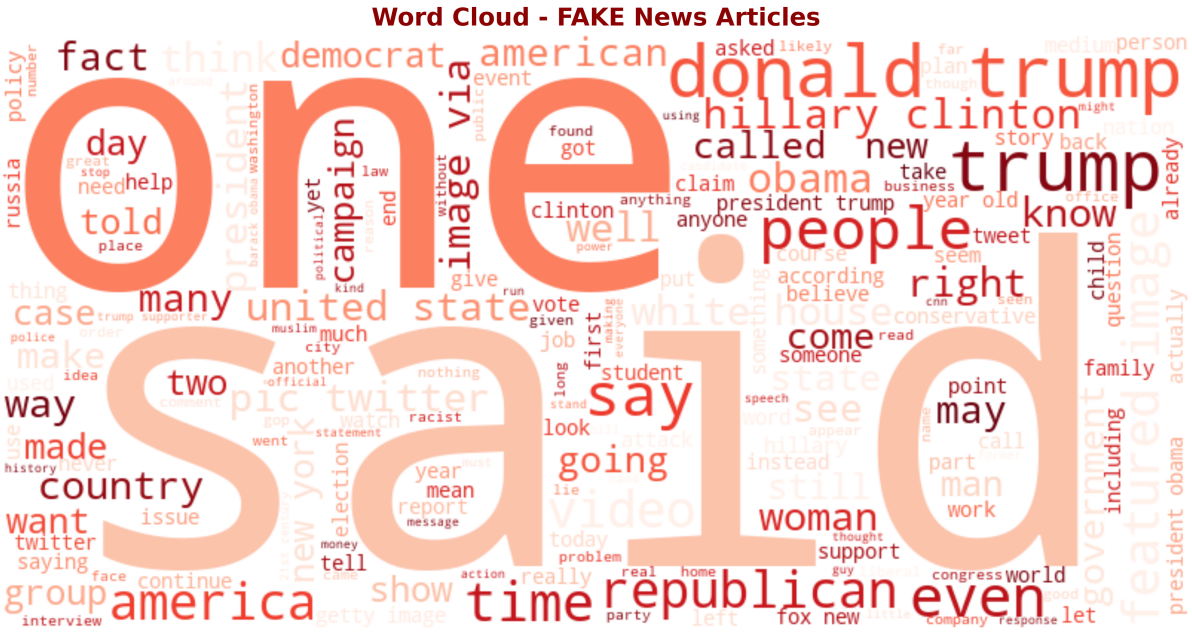


Fake News Detection System: Noisy Data Technical Report

Learning Curve



Word Cloud: Fake News



Fake News Detection System: Noisy Data Technical Report

Word Cloud: Real News

